

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

THIẾT KẾ KIẾN TRÚC HƯỚNG SỰ KIỆN

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Event-Driven Microservices Architecture

Asynchronous communication, Decoupled communication

Event Hubs

Stream-Processing

Real-time processing

High volume events

Problem: Handle Millions of Events Across Microservices

Considerations

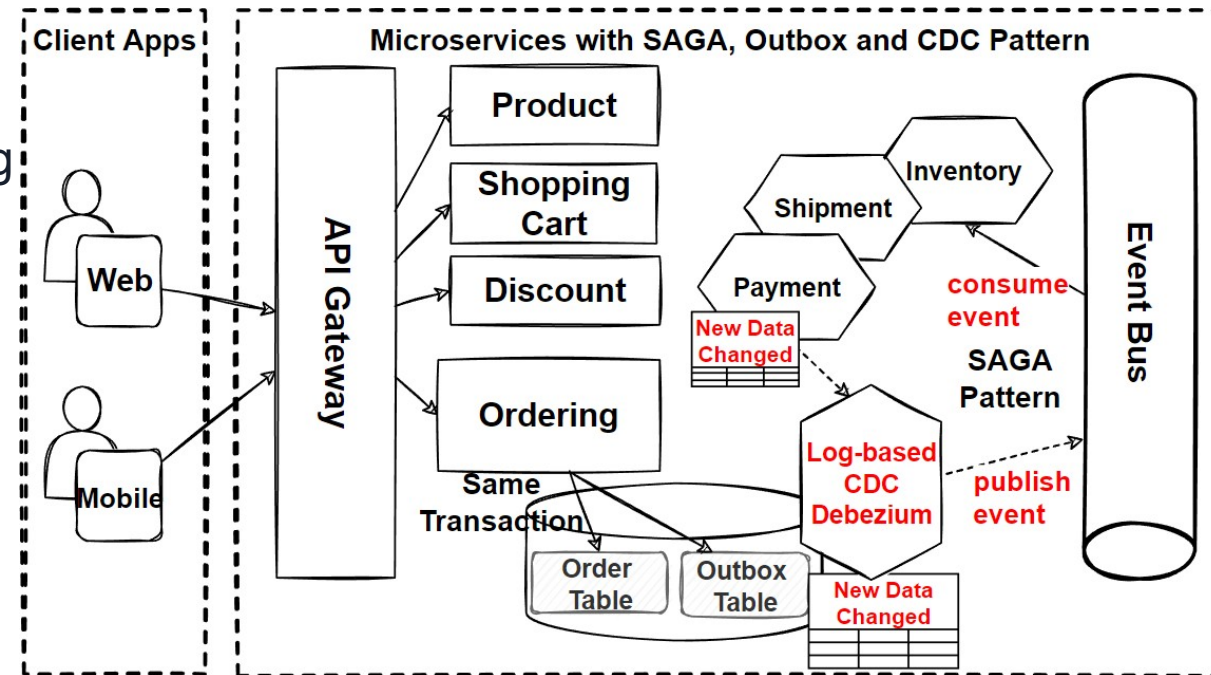
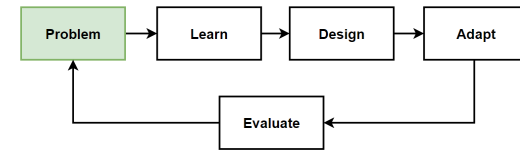
- What if we have thousands of microservices that need to communicate with millions of events ?
- If multiple subsystems must process the same events
- Required Real-time processing with minimum latency.
- Required complex event processing, like pattern matching
- Required process high volume and high velocity of data, i.e. IoT apps.

Problems

- Decoupled communications for thousands of microservices
- Real-time processing
- Handle High volume events

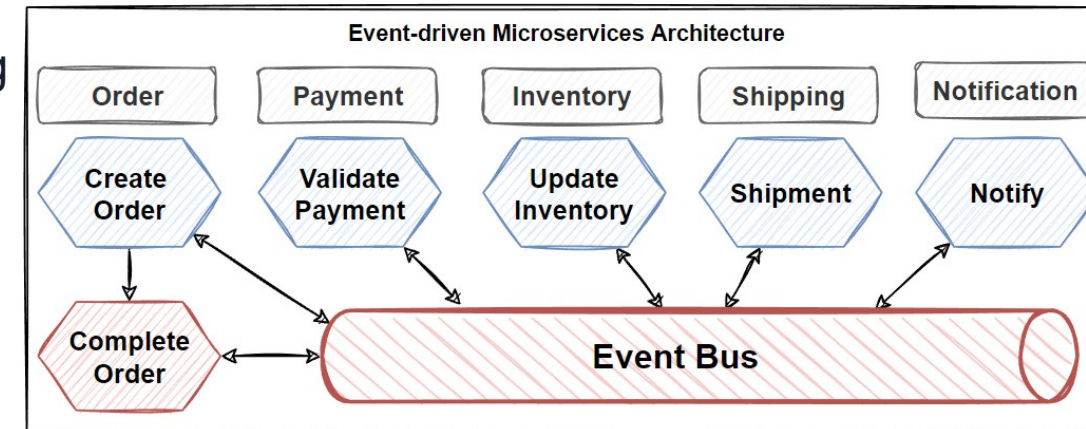
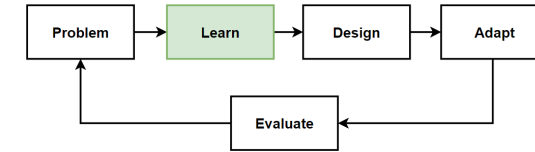
Solutions

- Event-driven architecture for microservices



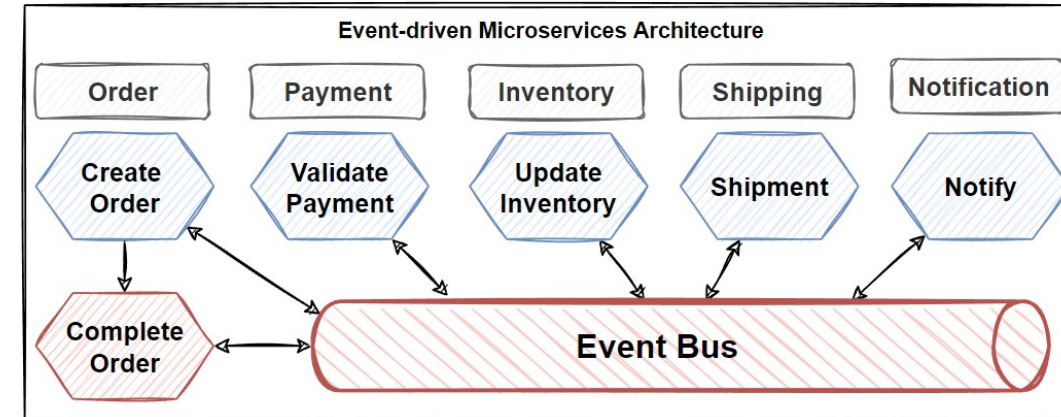
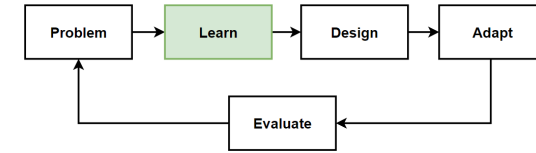
Introduction - Event-driven Architecture

- **Microservices** architectures are designed to be **highly modular** and **flexible**, that can be **scaled** and **managed separately** and use of **APIs** to communicate between services.
- **Event-driven architecture**, microservices can communicate by **publishing** and **subscribing** to **events**, than directly calling each other's APIs.
- **Event-driven microservice** architecture is means **communicating** with microservices via **event messages** and we can do **asynchronous behavior** and **loosely coupled**.
- **Instead of sending request** when data needed, services **consume** them **via events**.
- **Huge Innovations** on the Event-Driven Microservices Architectures;
- **Real-time messaging** platforms, **stream-processing**, **event hubs**, **real-time processing**, **batch processing**, **data intelligence**.



Event-Driven Microservices Architecture

- **Event-driven microservices architecture**, services communicate with each other by **publishing** and **subscribing** to **events**.
- When a service needs to communicate with another service, it **publishes an event** to a message queue or event bus. Other services can then **subscribe** to that **event** and take appropriate action when the **event** is **received**.
- **Asynchronous communication**
Allows services to communicate asynchronously. Service can publish an event and continue processing without waiting for a response from the other service.
- **Decoupled communication**
Decouples the publisher and subscriber, allows to evolve independently without affecting each other.



Event-Driven Microservices Architecture - 2

- **Real-time processing**

Support real-time processing, as events are published and consumed as soon as they occur. Need to react to events in real-time, such as in systems that use CDC to track changes to a database.

- **High volume events**

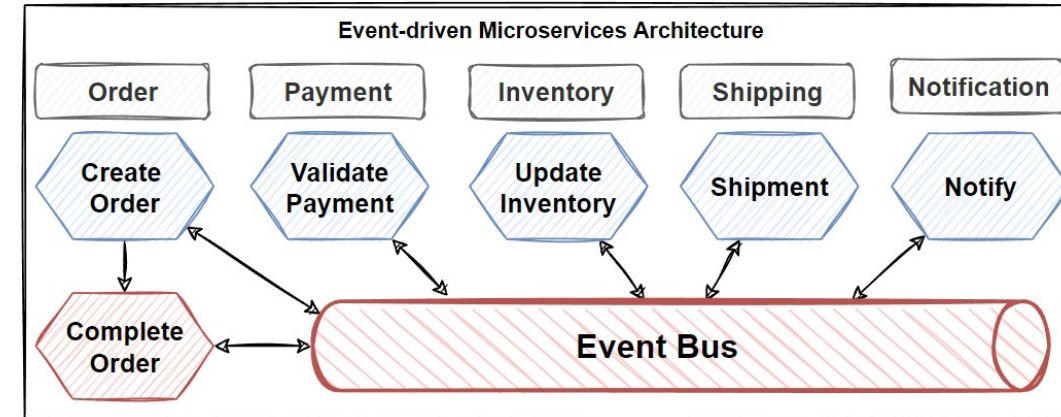
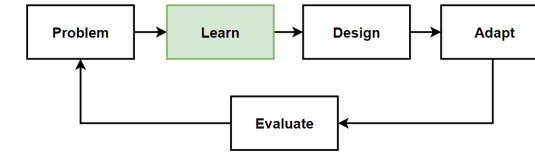
Well-suited to handling high volume events, as they can scale horizontally by adding more event consumers as needed. Can be scaled independently to handle increased load.

- **Responsible business capability**

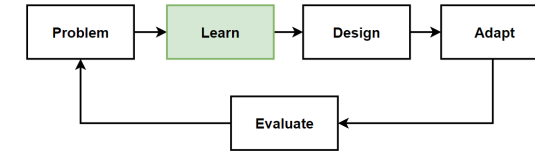
Each service is responsible for a specific function or business capability.

- Services communicate with each other by publishing and subscribing to events, that make it easier to build and maintain complex systems.

- Allows to work on different parts of the system in parallel without having to worry about the impact on other components.



Real-time Processing and High Volume Events in Event-Driven Microservices Architecture



- **Real-time processing**

Real-time processing is achieved by using a message queue or event bus to publish and consume events as they occur.

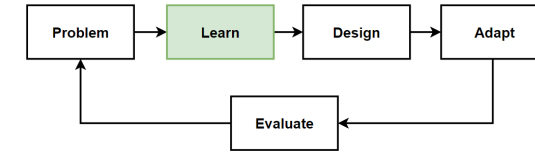
- When an event is generated, it is published to the message queue or event bus and made available to any interested subscribers.
- Allows you to react to events in real-time, as they are published and consumed as soon as they occur.
- When need to perform real-time analytics or trigger actions based on changes to the data.

- **High Volume Events**

Using a message queue or event bus that can handle high volumes of events and distribute them to multiple consumers.

- When you have a system that generates a large number of events, use a event bus to distribute those events to multiple consumers, that can process events in parallel.
- Allows you to scale up the number of event consumers as needed to handle increased load.
- Event-driven microservices architectures to process events in real-time and scale to handle high volumes of events.

Event Hubs and Event Streaming in Event-Driven Microservices Architecture



- **Event Hubs**

Act as a central hub for data ingestion and distribution, allowing microservices to publish and subscribe to data streams.

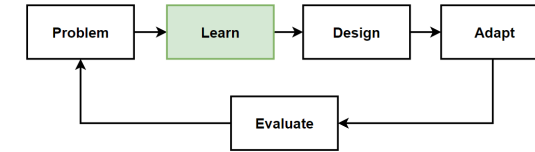
- Each microservice publish its data to an Event Hub. Other microservices can then subscribe to the Event Hub and consume the data as needed.
- Communicate with each other in real-time, that needs to be highly reliable and scalable.

- **Event Streaming**

Allows you to capture and process a stream of events in real-time. Publish and consume events as they occur.

- Allowing to build real-time data pipelines that can process and analyze data as it is being generated.
- One common use case for Event Hubs and event streaming in microservices architectures is real-time analytics.

Use Cases of Event Hubs and Event Streaming



- **Use Case – Customer Purchase on E-Commerce**

E-commerce website that generates a large number of events as customers browse and purchase products.

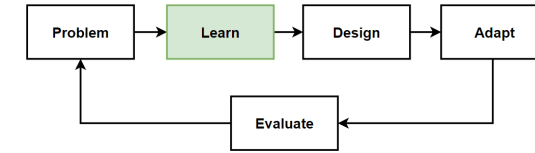
- Using event streaming, build a real-time analytics pipeline that processes these events as they occur and generates insights in real-time.
- This could include tracking customer behavior, identifying trends and patterns, and triggering actions based on changes to the data.

- **Use Case – Data Synchronization**

Different services may need to access the same data, and not be able to access each other's databases directly.

- Using event streaming, we can build a data synchronization pipeline that captures changes made to the data and streams them to other services in real-time. Ensures to access the most up-to-date data.
- Event Hubs and event streaming is a powerful tool for building real-time data pipelines in microservices architectures that provides flexible platform for data ingestion and distribution.

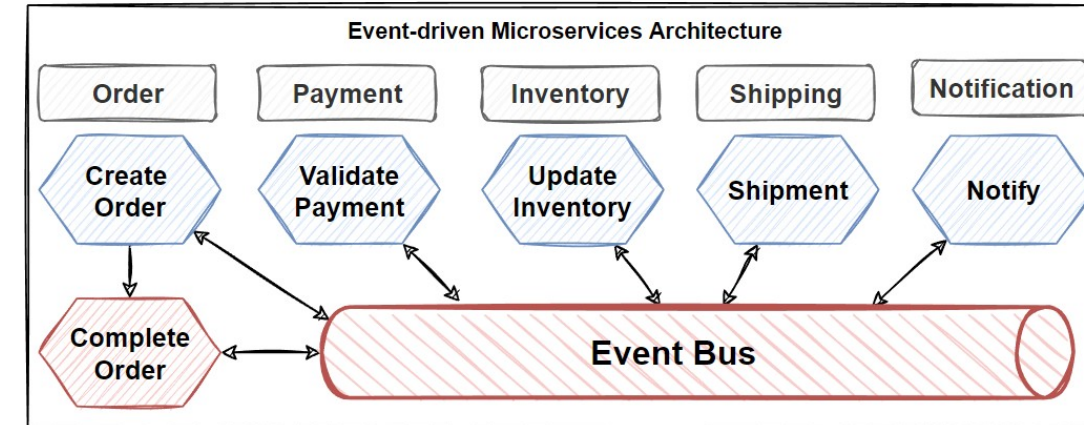
Real-world Examples of Event-Driven Microservices



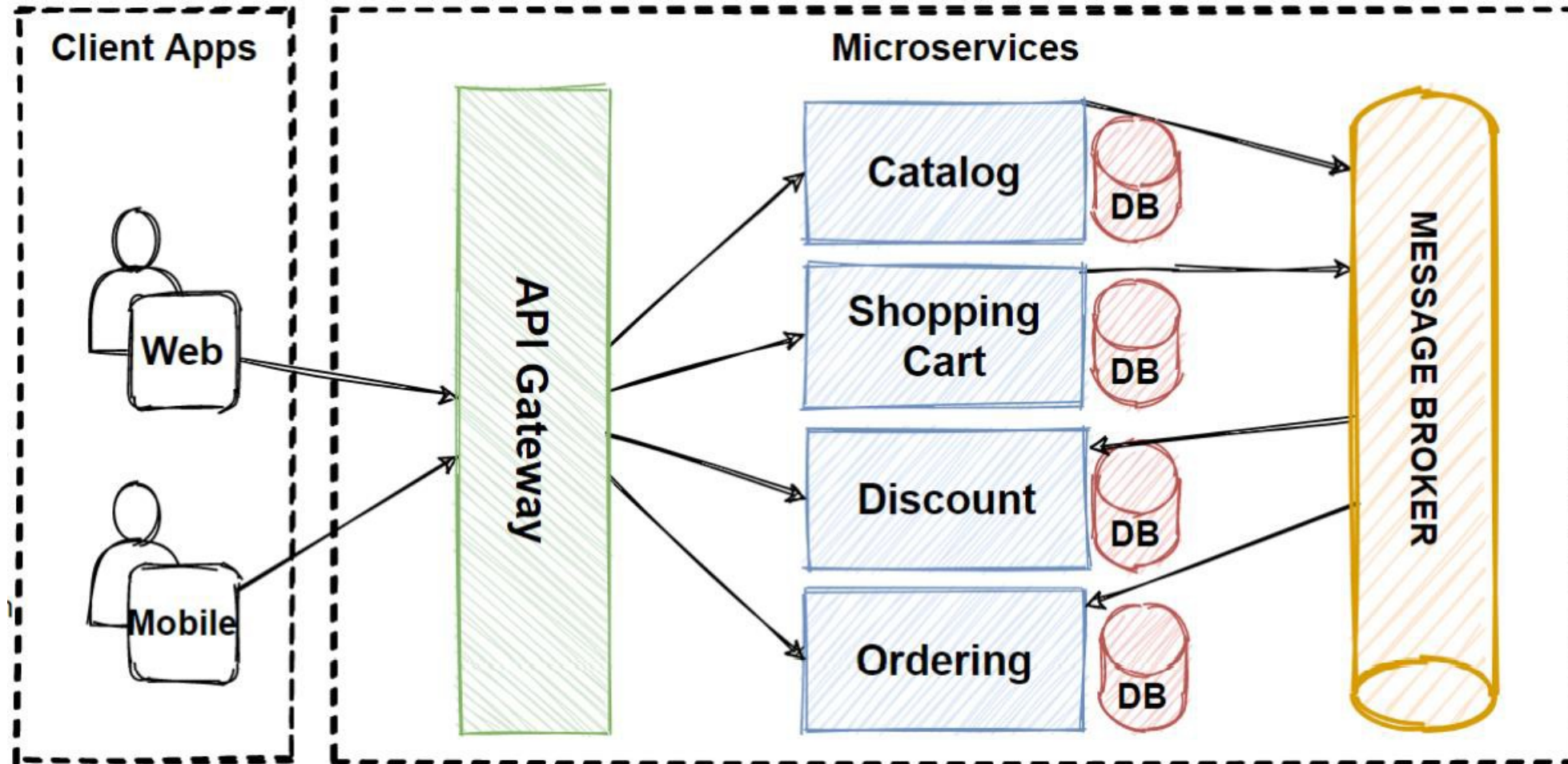
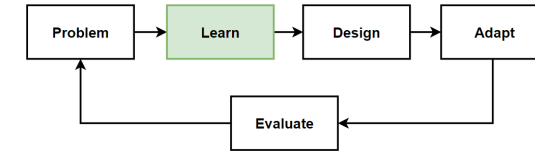
- **E-commerce application** we have a few microservices like: **customer, order, payment and products**.
- **Customer create orders** with some products and, if the payment is successful, the products should be delivered to the customer.

Events

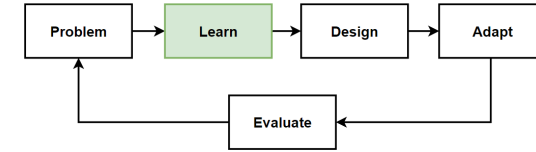
- a customer creates an order
 - the customer receives a payment request
 - if the payment is successful the stock is updated and the order is delivered
 - if the payment is not successful, rollback the order and set order status is not completed.
- This is more **humanly readable** and, if a new business requirement appears, it is **easier to change** the flow.
 - **Microservices** will **only care** about the **events**, not about the other microservices, **process only events** and **publish new event** to trigger other services.



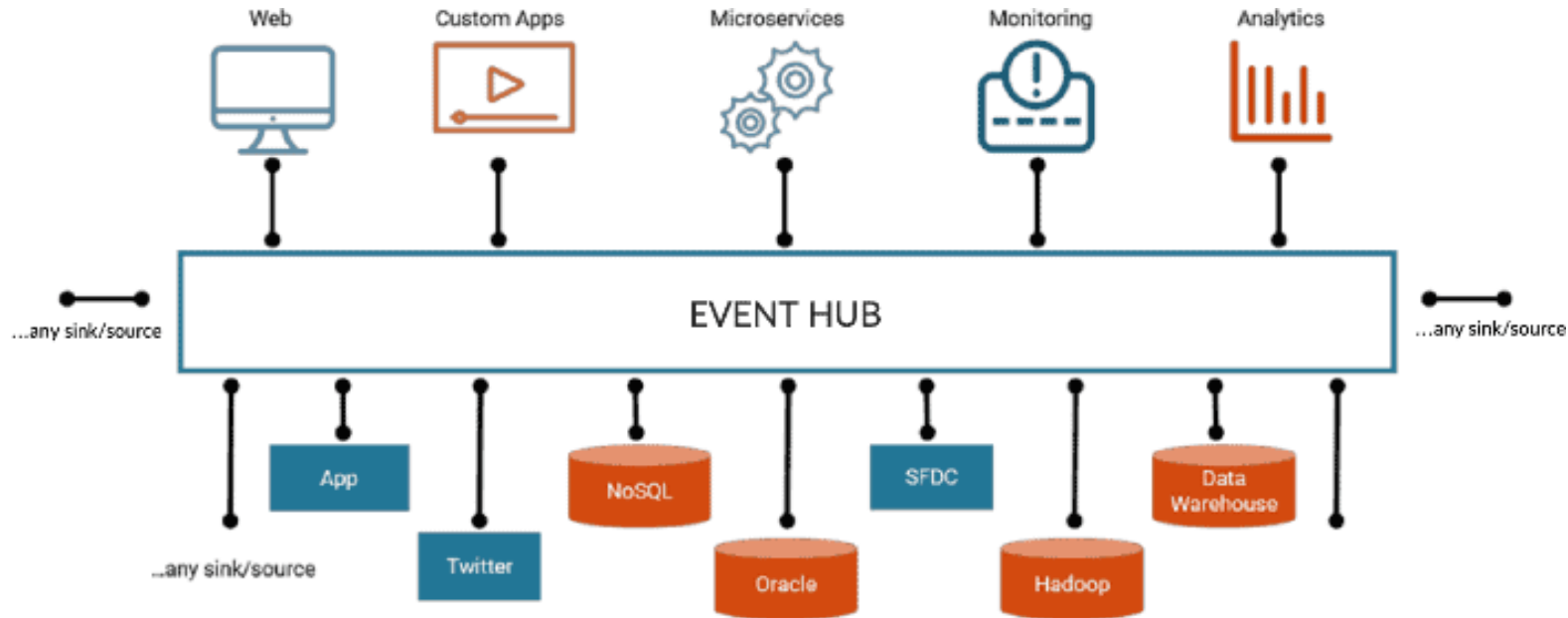
Traditional Event-Driven Microservices Architecture



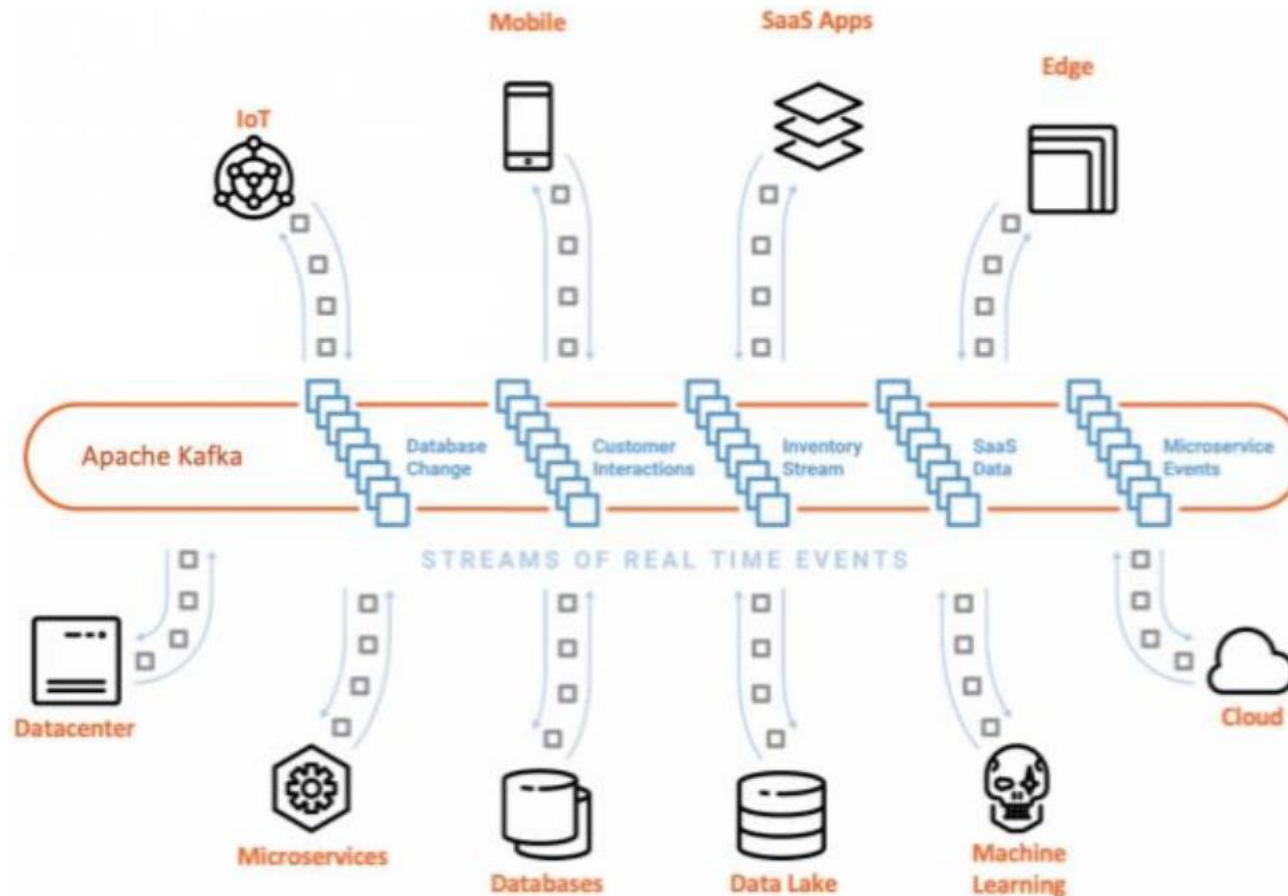
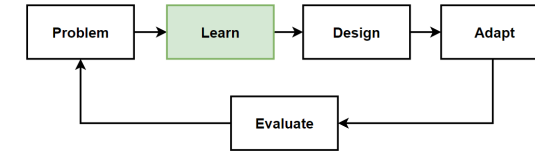
Evolved Event-Driven Microservices Architecture



- **Huge Innovations** on the Event-Driven Microservices Architectures.
- **Real-time messaging platforms, stream-processing, event hubs, real-time processing, batch processing, data intelligence.**
- **Event-Hubs** is huge event store database that can make **real-time processing**.



Kafka Event-Driven Microservices Architecture



- Global-scale
- Real-time
- Persistent Storage
- Stream Processing



Before Design – What we have in our design toolbox ? - Old

Architectures Patterns&Principles

- **Microservices Architecture**
 - The Database-per-Service Pattern
 - Polygot Persistence
 - Decompose services by scalability
 - The Scale Cube
 - Microservices Decomposition Pattern
 - Microservices Communications Patterns
 - Microservices Data Management Patterns
 - **Microservices Distributed Transaction Pattern**

Microservices Data Choosing Database

- The Shared Database Anti-pattern, Relational and NoSQL Databases
- CAP Theorem—Consistency, Availability, Partition Tolerance
- Data Partitioning: Horizontal, Vertical and Functional Data Partitioning
- Database Sharding Pattern

Microservices Data Commands&Queries

- Materialized View Pattern
- CQRS Design Pattern
- Event Sourcing Pattern
- Eventual Consistency

Microservices Distributed Transactions

- **SAGA Pattern**
- **Choreography and Orchestration-based SAGA**
- **Compensating Transaction Pattern**
- **Dual-Write Problem**
- **Transactional Outbox Pattern**
- **CDC - Change Data Capture**

FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

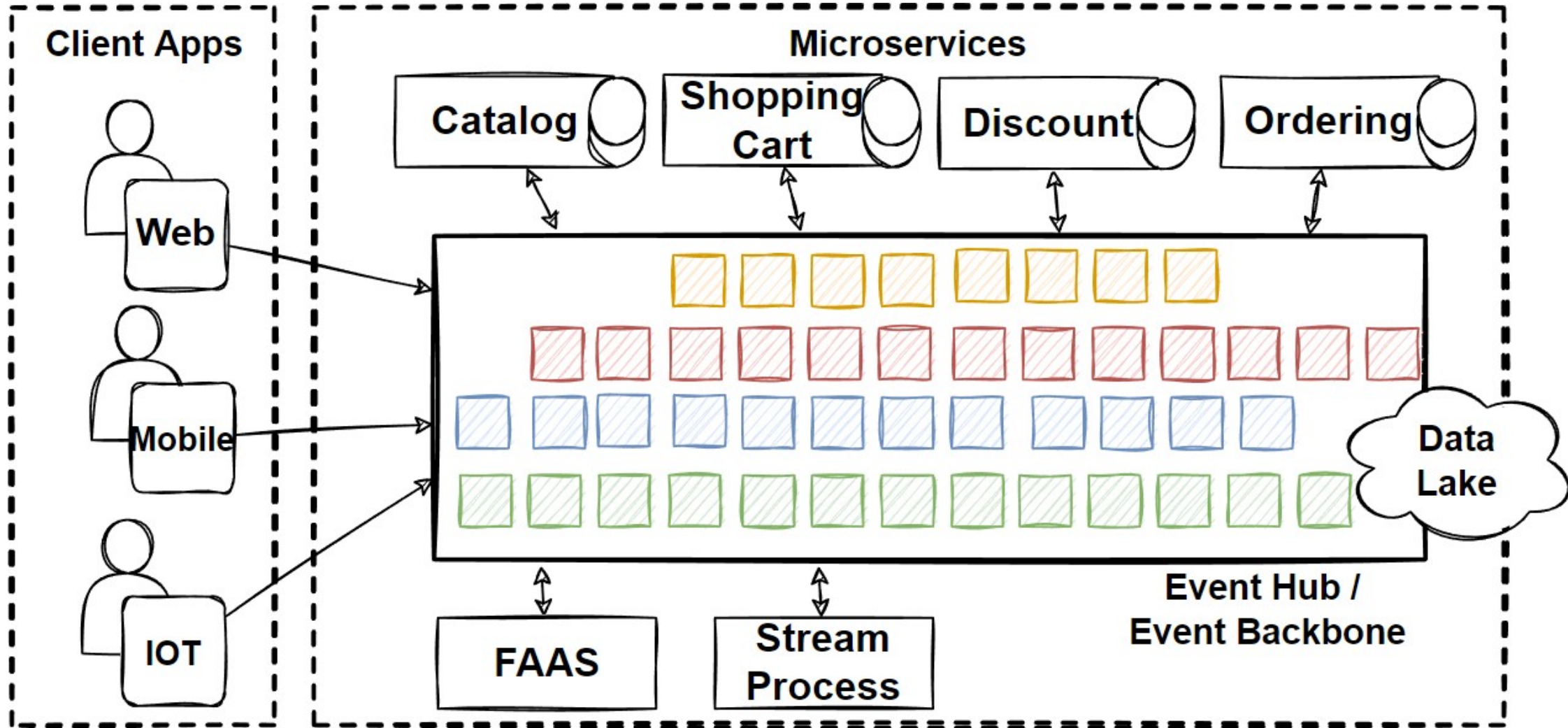
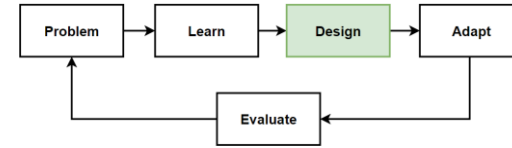
Non-FR

- High Scalability
- High Availability
- Millions of Concurrent User
- Independent

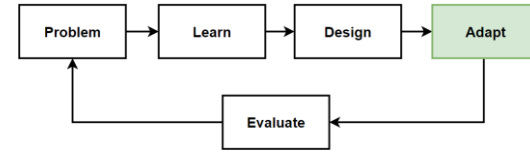
Before Design – What we have in our design toolbox ? - New

Architectures	Patterns&Principles	Microservices EDA	Non-FR	FR
• Microservices Architecture • Event-Driven Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns • Microservices Data Management Patterns • Event-Driven Architecture	• Asynchronous, Decoupled communication • Event Hubs • Stream-Processing • Real-time processing • High volume events	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

Event-Driven Microservices Architecture



Adapt: Event-Driven Microservices Architecture



Frontend SPAs

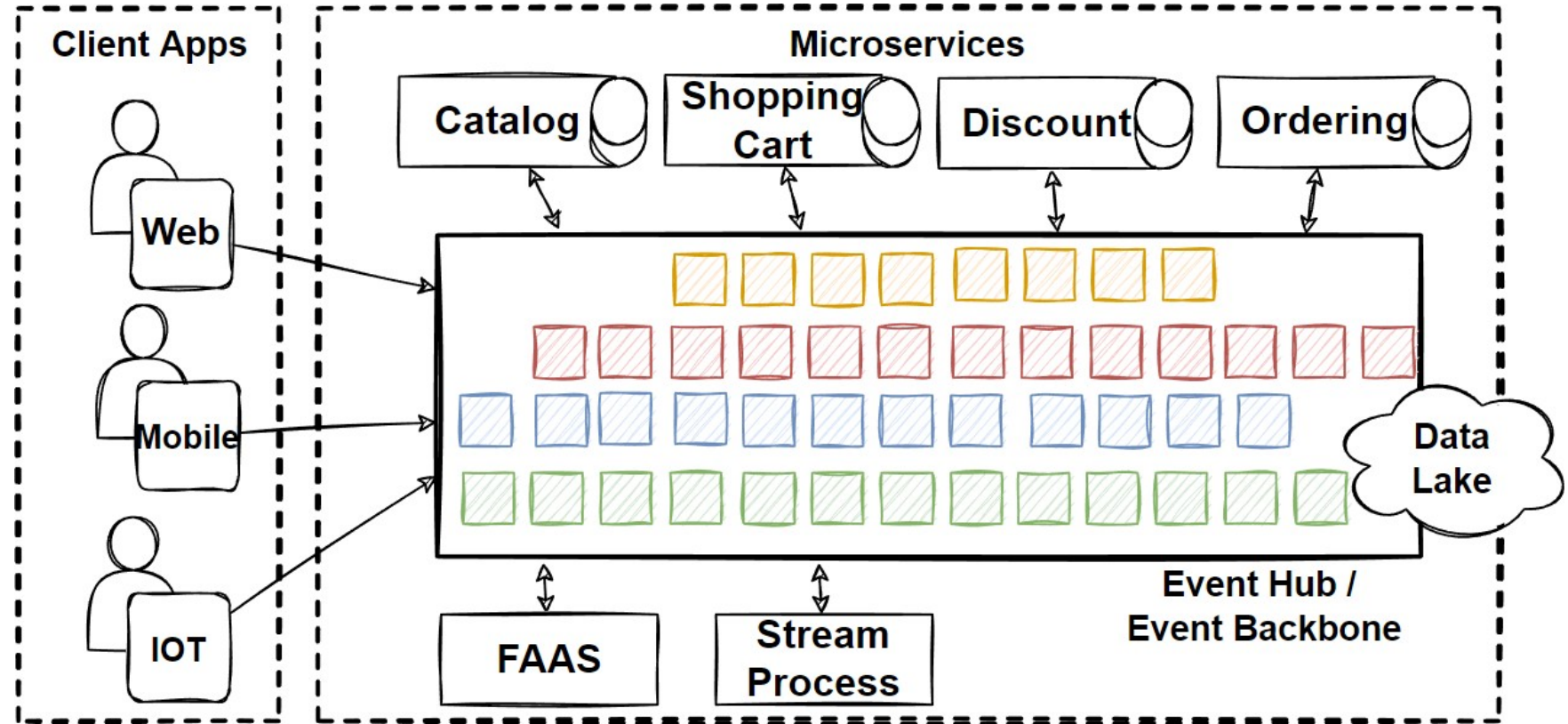
- Angular
- Vue
- React

API Gateways

- Kong Gateway
- Express Gateway
- Amazon AWS API Gateway

Backend Microservices

- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS



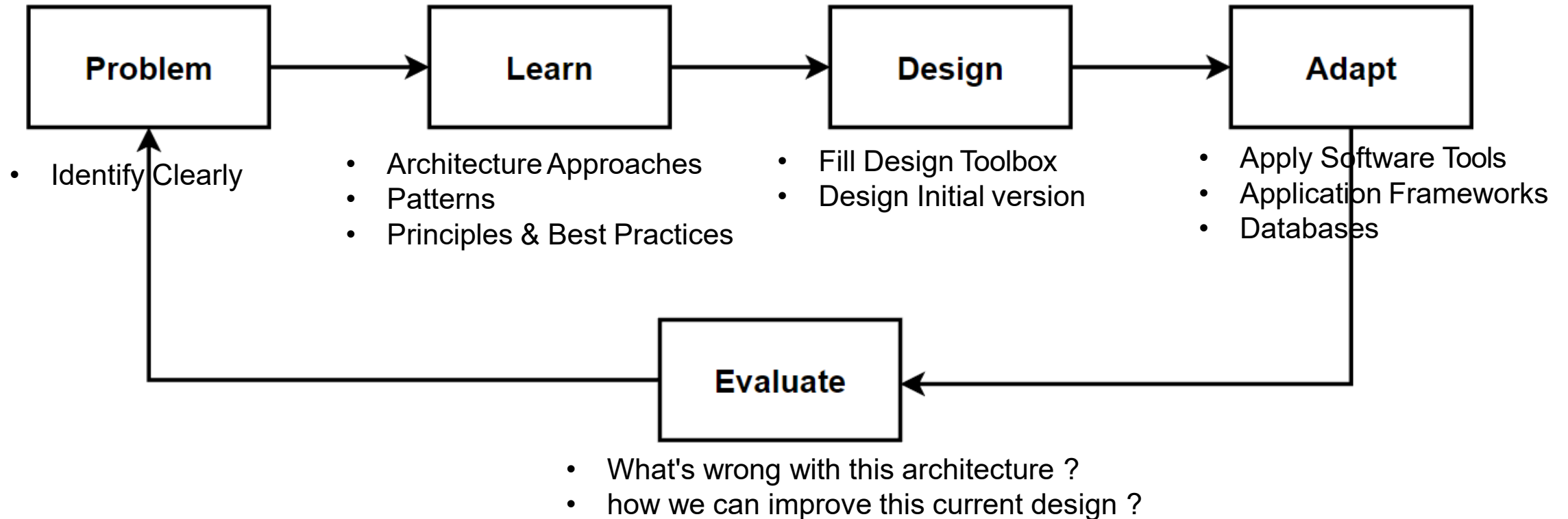
Event Hubs

- Apache Kafka
- Apache Spark

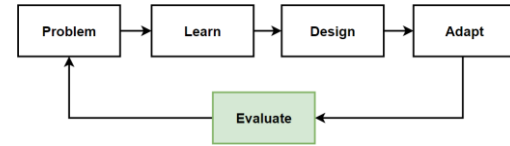
Cloud Event Hubs

- Azure Event Hubs
- AWS Kinesis
- Oracle Event Hubs

Way of Learning – The Course Flow



Evaluate: Event-Driven Microservices Architecture

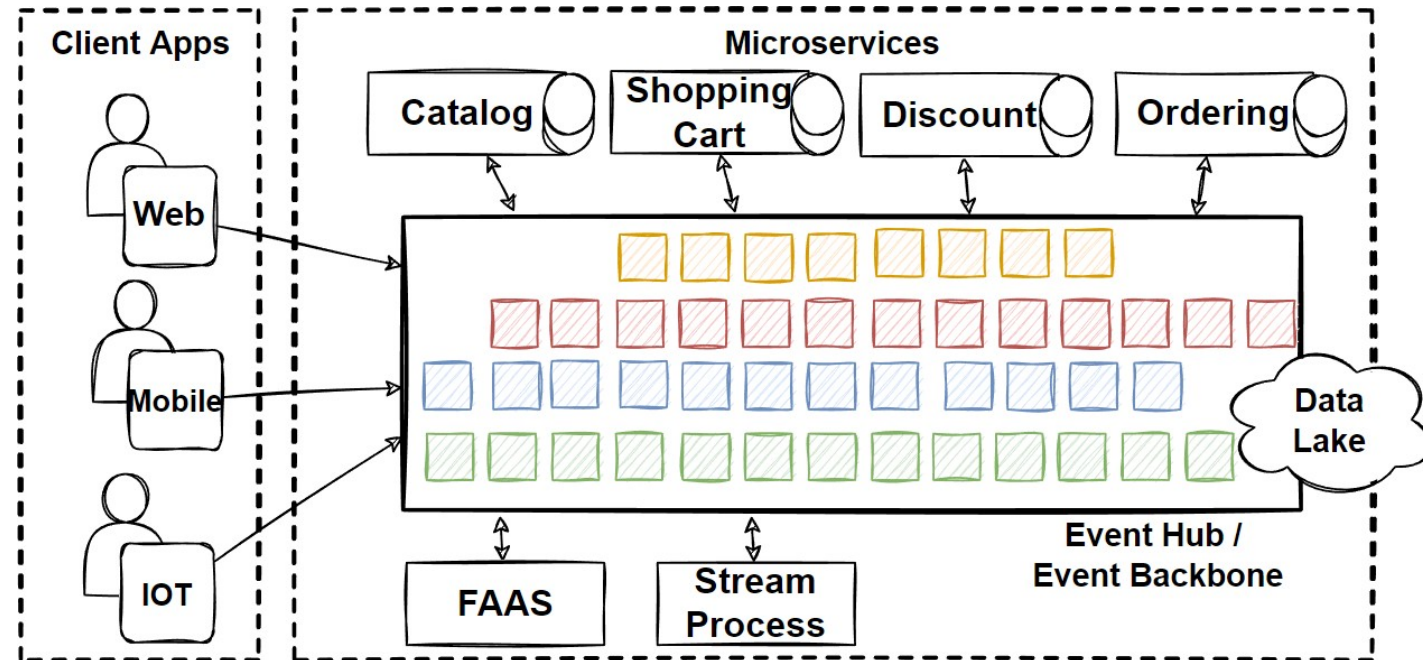


Benefits

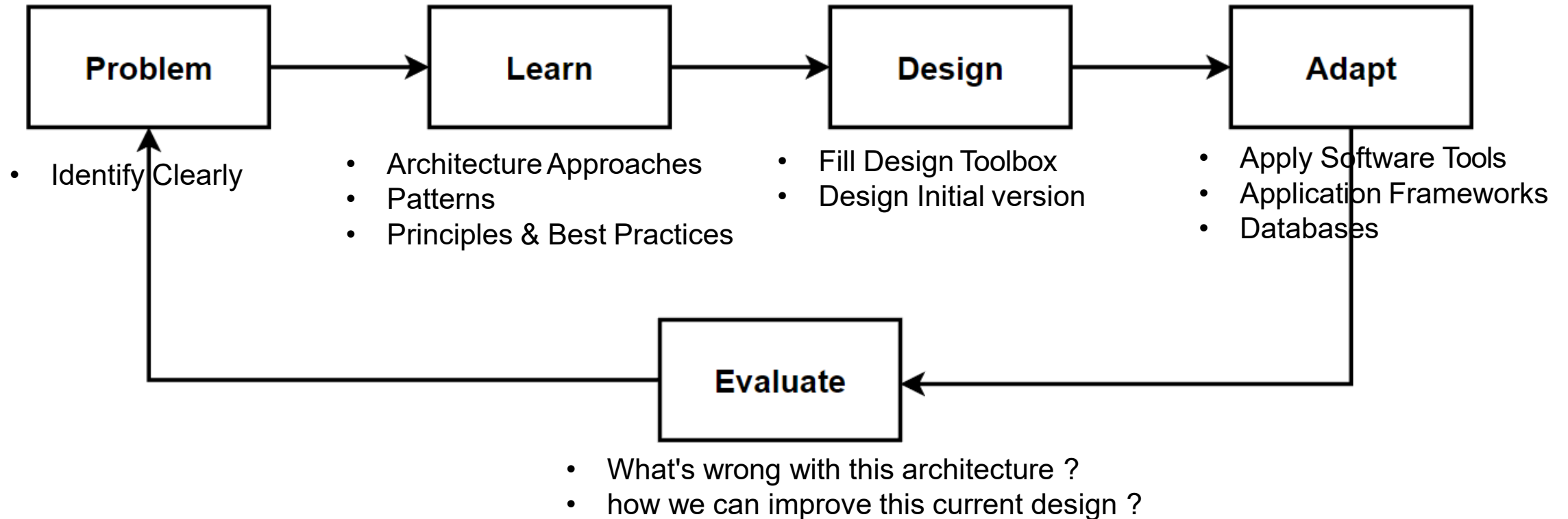
- Event Streaming
- Real-time Processing
- High Volume Events
- Decouple services, increased Scalability
- Resilience

Drawbacks

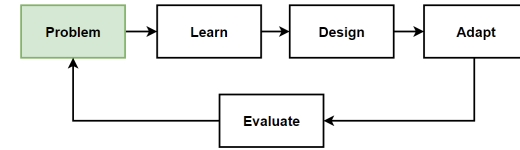
- Increased Complexity, event-driven makes your system more complex design.
- Hard to Debugging
- Add latency to the event publishing process
- Integration into Distributed Transactions
- Difficult to set up and maintain



Way of Learning – The Course Flow



Problem: Database operations are expensive, low performance



Considerations

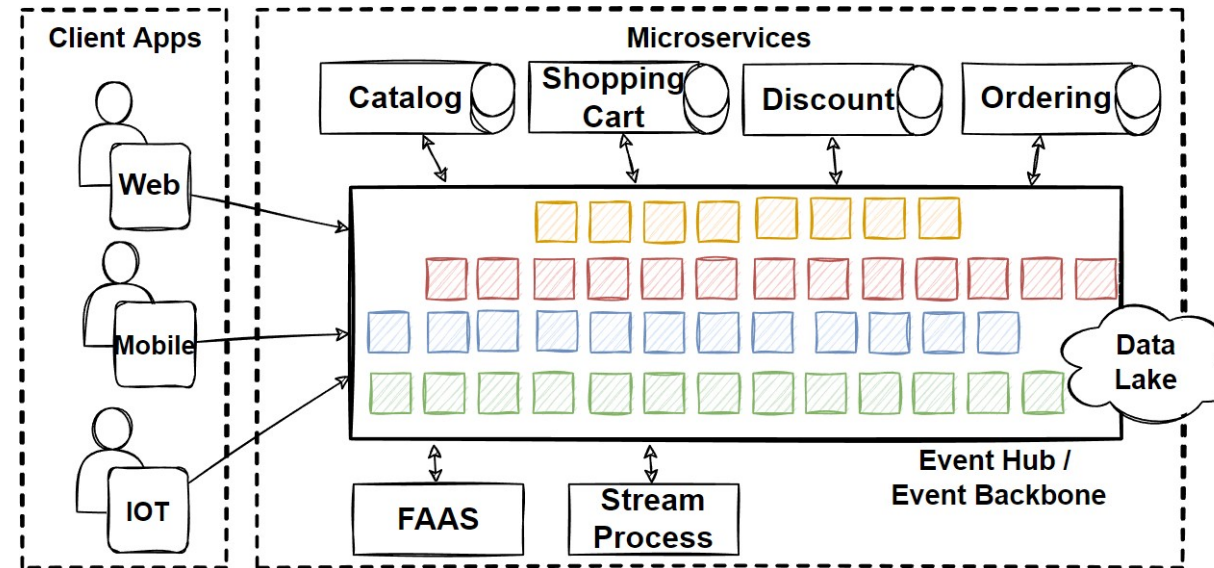
- Event-driven architecture comes with latency when publishing and subscribing events from the Event Hub.
- Sync REST APIs communication make expensive calls to a database that reduce performance.
- How can we make more faster that increase performance of communications in Microservices Architecture ?

Problems

- Slowness and Low Performance Communication
- Latency when publishing and subscribing events
- Rest APIs make Database calls that are expensive, low performance

Solutions

- Distributed cache
- Storing frequently accessed data in a distributed cache



**CÁM ƠN ĐÃ CHÚ Ý
LẮNG NGHE!**